

Guia de estudio - Sistema de Gestion Academica

Proyecto de Programacion IV

Este documento explica como esta organizado el proyecto SGA, que hace cada archivo, como viajan los datos y que diferencias existen entre el frontend actual y el backend en desarrollo.

1. Vista general

El sistema es una aplicacion web para administrar alumnos y docentes. La portada permite entrar a cada modulo. El modulo de alumnos permite alta, consulta, edicion y baja. El modulo de docentes ofrece las mismas operaciones con el campo especialidad.

El frontend funciona actualmente sin depender del servidor: guarda los datos en localStorage, que pertenece al navegador. El backend es una API Express separada que, por ahora, guarda alumnos en un arreglo de memoria.

2. Estructura del proyecto

SGA/

Frontend/ Paginas, estilos y logica del navegador

index.html Portada y accesos a los modulos

alumnos.html Formulario y tabla de alumnos

docentes.html Formulario y tabla de docentes

CSS/estilos.css Estilos compartidos

JS/storage.js Utilidades de localStorage

JS/ui.js Mensajes, escape HTML y correo

JS/alumnos.js Logica CRUD de alumnos

JS/docentes.js Logica CRUD de docentes

JS/asincronia.js Ejercicios de Promises y fetch

Backend/ API REST con Node.js y Express

server.js Arranque de Express

routes/ URLs y metodos HTTP

controllers/ Funciones que responden solicitudes

data/ Arreglo que simula una base de datos

3. Como se carga una pagina

Cada HTML define la estructura visual y carga estilos.css. En alumnos.html y docentes.html los scripts tienen defer: el navegador descarga los archivos y espera a que el HTML este disponible antes de ejecutarlos.

El orden declarado es importante. storage.js define guardarDatos y obtenerDatos; ui.js define mostrarMensaje, escaparHTML y esCorreoValido; por ultimo alumnos.js o docentes.js utiliza esas funciones compartidas.

storage.js -> ui.js -> alumnos.js

storage.js -> ui.js -> docentes.js

4. Persistencia con localStorage

con `JSON.stringify`. `obtenerDatos` recupera el texto con `getItem` y lo convierte de nuevo con `JSON.parse`. Si no existe la clave, devuelve un arreglo vacío.

```
guardarDatos("alumnos", alumnos)
```

```
const alumnos = obtenerDatos("alumnos")
```

Las claves `alumnos` y `docentes` mantienen separadas las dos colecciones. Los datos sobreviven a una recarga y al cierre del navegador, pero no se comparten con otros navegadores ni con el backend.

5. Modulo de alumnos

`alumnos.html` contiene un formulario con nombre, carrera y correo, y una tabla cuyo `tbody` tiene el `id` `listaAlumnos`. La tabla comienza vacía porque `alumnos.js` crea sus filas en tiempo de ejecución.

Al iniciar, `mostrarAlumnos(obtenerAlumnos())` lee la lista guardada y la renderiza. Para cada objeto genera una fila con ID, nombre, carrera, correo y botones de editar y eliminar.

Alta de alumno

El evento `submit` cancela el envío tradicional con `preventDefault`. Luego `trim` elimina espacios externos y se validan campos obligatorios, longitud mínima de nombre y carrera y formato del correo.

```
const alumno = { id: Date.now(), nombre, carrera, correo }
```

Si no hay una edición activa, el objeto se agrega con `push`. La lista completa se guarda de nuevo y la tabla se actualiza. Finalmente se reinicia el formulario.

Edicion y eliminacion

Al pulsar editar, `editarAlumno` busca el ID, completa los inputs y guarda `alumnoEditandId`. El siguiente `submit` modifica ese objeto en lugar de crear uno nuevo. `alumnoEditar` permite detectar el caso en que no se cambió ningún dato.

Al pulsar eliminar, el programa pide confirmación. `filter` crea una lista sin el ID elegido, la persiste y vuelve a renderizar la tabla. Si el registro estaba en edición, también se vacía el formulario.

6. Modulo de docentes

`docentes.js` repite el mismo patrón de `alumnos`, pero utiliza la clave `docentes` y el campo `especialidad`. Sus funciones principales son `obtenerDocentes`, `mostrarDocentes`, `editarDocente` y `eliminarDocente`.

La duplicación es comprensible mientras se aprende CRUD. A futuro se podría extraer una función genérica, pero conviene priorizar primero una API común y validaciones consistentes.

7. Utilidades de ui.js

`mostrarMensaje` escribe un texto en `#mensaje`, aplica una clase y lo oculta luego de 3000

escaparHTML coloca el texto en textContent y devuelve innerHTML. Esto evita que un dato ingresado por una persona se interprete como etiquetas o script cuando se inserta en la tabla.

esCorreoValido utiliza una expresion regular basica: exige texto antes y despues de @ y un punto en el dominio. Es una validacion practica del lado del cliente, no una verificacion de existencia del correo.

8. CSS y experiencia visual

estilos.css define el fondo, encabezado, navegacion, formularios, botones, tablas, tarjetas y mensajes. La media query de 700px cambia el menu y los formularios a una disposicion mas adecuada para pantallas pequenas.

Los iconos de acciones provienen de Font Awesome mediante CDN. Para usar la aplicacion con internet limitada, se podria instalar Font Awesome localmente o reemplazar esos iconos por texto accesible.

9. Backend Express

server.js crea la aplicacion Express, habilita express.json y monta las rutas de alumnos bajo el prefijo /alumnos. Luego escucha en el puerto 3000.

GET /alumnos lista todos los alumnos

GET /alumnos/:id busca un alumno por ID

POST /alumnos recibe un alumno en req.body

PUT /alumnos/:id actualiza un alumno

DELETE /alumnos/:id elimina un alumno

El controlador contiene las consultas GET. data/alumnos.js exporta un arreglo con Ana y Jose, que funciona como fuente temporal de datos mientras no existe MongoDB.

10. Problemas actuales del backend

Hay dos errores que deben corregirse antes de probar POST y DELETE: routes/alumnos.routes.js usa alumnos pero no importa ese arreglo desde data/alumnos.js; ademas data/alumnos.js exporta una constante y DELETE intenta reasignarla con alumnos = filter(...).

Tambien conviene validar que un ID exista antes de acceder a sus propiedades y devolver estados HTTP apropiados, por ejemplo 404 cuando no se encuentra un alumno y 201 al crear uno.

11. Como ejecutar

cd Backend

npm install

npm start

Con el servidor iniciado, la API queda disponible en http://localhost:3000. Para probarla se pueden usar las solicitudes de pruebas del archivo Backend/pruebas.http si el editor tiene una extension HTTP instalada.

El frontend puede abrirse desde Frontend/index.html. Para evitar restricciones del navegador al usar fetch en el futuro, sera conveniente servir Frontend con un servidor local y configurar CORS en Express.

12. Flujo completo de un alta

1. La persona completa el formulario.
2. submit evita recargar la pagina.
3. Se limpian y validan los valores.
4. Se crea o modifica un objeto JavaScript.
5. JSON.stringify guarda la lista.
6. La tabla se regenera con innerHTML.
7. Se muestra un mensaje y se limpia el formulario.

13. Proximos pasos sugeridos

1) Arreglar las rutas POST y DELETE del backend. 2) Conectar alumnos.js y docentes.js con fetch. 3) Agregar CORS y manejo de errores. 4) Persistir en MongoDB. 5) Incorporar autenticacion. 6) Crear los modulos de cursos y materias. 7) Agregar pruebas para validaciones y endpoints.

14. Ideas clave para estudiar

HTML define estructura; CSS define presentacion; JavaScript agrega comportamiento. Los eventos conectan acciones de la persona con funciones. JSON permite transportar estructuras como texto. localStorage persiste en el navegador. Express recibe solicitudes HTTP y devuelve respuestas JSON. Un CRUD representa crear, leer, actualizar y eliminar.